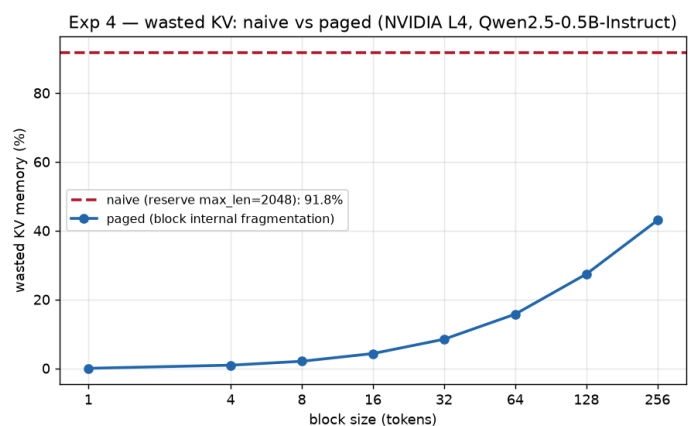
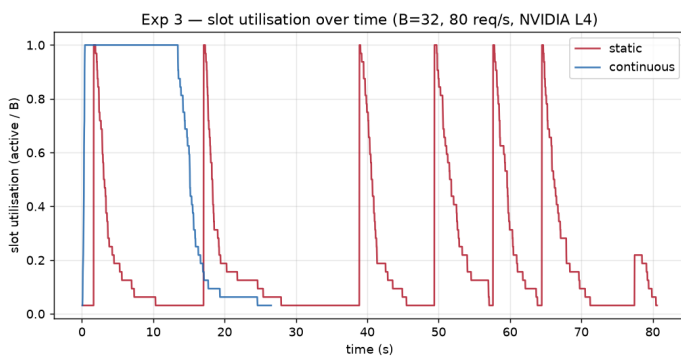
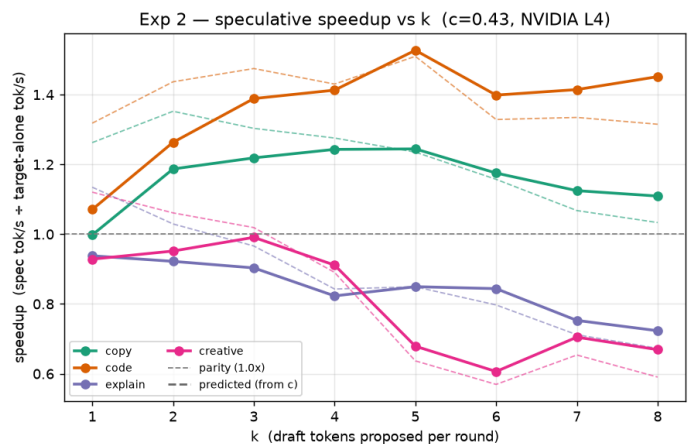
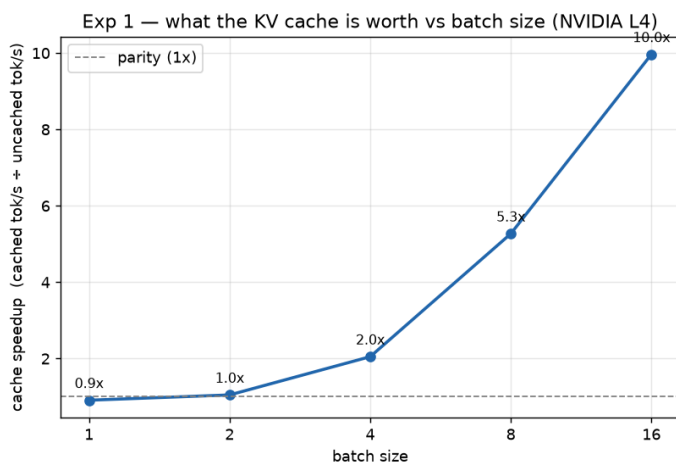


What Each Inference Trick Is Worth

One pipeline, four controlled experiments — what each serving optimization is really worth on an NVIDIA L4.

All numbers are on NVIDIA L4 (bf16), synchronised, warmed up, and repeated with their spread. Trust: a validation gauntlet (8/8 pass) shows a single un-synchronised matmul under-reports its time by 334x, so every timer here calls `cuda.synchronize()`; the CUDA-event and `perf_counter` clocks agree to 0.7%, and measured FP32 throughput stays under the card's roofline. Both sides of every comparison were checked to emit the same tokens before any speedup was claimed.

Exp	Toggle	Headline measurement
1 KV cache	recompute vs cache	batch 1: 0.91x (60x more math, same time); batch 16: 10.0x
2 Spec decode	target alone vs draft+verify	c=0.43 (draft launch-bound); best speedup 1.53x; exact modulo bf16 ties
3 Cont. batching	static vs continuous	continuous 7.5 vs static 2.5 req/s; p95 14s vs 68s
4 PagedAttention	reserve-max vs paged blocks	naive waste 92%; paging fits ~12x more seqs



Exp 1 Expected the cache to be a big win everywhere.

At batch 1 it is a wash (0.99x) — the uncached loop does 60x more arithmetic but the GPU was memory-bound and idle, so the extra math is free; the cache only wins as the batch fills the ALUs.

Exp 2 Expected a 15x-smaller draft to give a big speedup.

Naive it was a slowdown: the 0.5B draft is kernel-launch-bound (step time tracks layers, not params), so $c=0.43$ and no acceptance rate can win. The diagnosis, not the mechanism, is the work.

Exp 3 Expected static batching to be slow.

It is not slow, it is idle: the batch drains to almost empty while one long request finishes, and continuous batching holds the slots full for ~3x the throughput.

Exp 4 Expected paging to save some memory.

Naive reserving `max_len` wastes 92% of KV against a real length mix; paging fits ~12x more sequences, and that concurrency is where memory becomes speed.

The one measurement that changed how I think about serving:

At batch 1 the uncached loop computes 60x more token positions than the cached loop and finishes in the same wall-clock time. Serving a single stream is memory-bound: the arithmetic units sit idle waiting on weights, so 'wasted' compute is free — until you add enough concurrent sequences that arithmetic becomes the wall. Every one of these four techniques is really about the same thing: keeping the expensive, bandwidth-limited GPU busy with useful work instead of waiting or reserving.